

0. 问题定义与计算目标

设 A 一次提交 m 个查询，每个查询有 d 维特征；B 有 L 个候选。把查询和候选分别按行排列：

$$Q = \begin{bmatrix} q_0^T \\ \vdots \\ q_{m-1}^T \end{bmatrix} \in \mathbb{R}^{m \times d}, \quad C = \begin{bmatrix} c_0^T \\ \vdots \\ c_{L-1}^T \end{bmatrix} \in \mathbb{R}^{L \times d}.$$

理想输出不是一个数，而是一张 $m \times L$ 的分数表：

$$S = QC^T \in \mathbb{R}^{m \times L}, \quad S_{q,t} = \langle q_q, c_t \rangle = \sum_{f=0}^{d-1} Q_{q,f} C_{t,f}.$$

打包问题。 CKKS 的一个密文只有一维槽位。我们要构造一个一维索引，使每个槽位唯一代表一个二维组合 (q, t) ，并保证这个槽位内的逐特征乘加恰好得到 $S_{q,t}$ 。

为避免“维度在推导中悄悄变化”，先固定所有符号：

对象	形状	含义
Q	$m \times d$	A 的查询特征矩阵
C	$L \times d$	B 的完整候选库
S_{slots}	标量	一个 CKKS 密文的可用槽位数，当前为 4096
$T = \lfloor S_{\text{slots}}/m \rfloor$	标量	每个查询在同一密文中可分到的候选位置数
$C^{(j)}$	$T \times d$	第 j 个候选 tile；最后一块不足 T 时补零
$A^{\text{pack}}, B_{\text{pack}}^{(j)}$	$mT \times d$	查询与候选在加密前完成对齐后的二维矩阵

后面始终使用 query-major 顺序：先放完 q_0 的 T 个候选位置，再放 q_1 的 T 个候选位置。

1. 修改前的逐候选轮询计算

旧方案只沿“查询方向”使用槽位。对每个特征 f ，A 加密长度为 m 的列向量：

$$\mathbf{x}_f = Q_{:,f} \in \mathbb{R}^m, \quad \text{Enc}(\mathbf{x}_f).$$

当 B 处理第 t 个候选 c_t 时，它把同一个候选特征值广播到全部查询槽位：

$$\mathbf{y}_f^{(t)} = C_{t,f} \mathbf{1}_m = [C_{t,f}, \dots, C_{t,f}]^T \in \mathbb{R}^m.$$

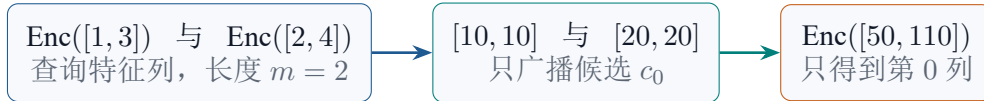
于是一次逐槽乘加只能生成候选 c_t 对应的一列：

$$\mathbf{z}^{(t)} = \sum_{f=0}^{d-1} \text{Enc}(\mathbf{x}_f) \odot \mathbf{y}_f^{(t)} = \text{Enc} \left(\begin{bmatrix} \langle q_0, c_t \rangle \\ \vdots \\ \langle q_{m-1}, c_t \rangle \end{bmatrix} \right) \in \text{Enc}(\mathbb{R}^m).$$

形状结论。 $\mathbf{x}_f$ 和 $\mathbf{y}_f^{(t)}$ 都只有 m 个有效位置，所以结果也只有 m 个分数；这 m 个分数共享同一个候选编号 t 。要得到完整 $m \times L$ 矩阵，只能令 $t = 0, 1, \dots, L-1$ 逐列轮询。

用 $m = 2, d = 2$ 的例子看得最清楚：

$$q_0 = [1, 2], \quad q_1 = [3, 4], \quad c_0 = [10, 20].$$



随后仍要重复：

$$c_0 \longrightarrow c_1 \longrightarrow c_2 \longrightarrow \dots \longrightarrow c_{L-1}.$$

轮询开销。简单点积阶段需要 Ld 次密文-明文逐槽乘法，并返回 L 个“长度为 m 的结果密文”。当 $m = 200, L = 483$ 时，每个结果只使用 $200/4096 = 4.88\%$ 的槽位。

2. 查询向量的槽位扩展

一个密文有 S_{slots} 个槽位。若一次处理 m 个查询，则每个查询最多可以拥有

$$T = \left\lfloor \frac{S_{\text{slots}}}{m} \right\rfloor$$

个连续位置。定义二维组合到一维槽位的映射：

$$\boxed{\text{slot}(q, t) = qT + t}, \quad 0 \leq q < m, \quad 0 \leq t < T.$$

对 $m = 2, T = 3$ ，一维槽位的语义不是六个无关数字，而是：

$$\underbrace{(q_0, c_0), (q_0, c_1), (q_0, c_2)}_{q_0 \text{ 的候选块}}, \quad \underbrace{(q_1, c_0), (q_1, c_1), (q_1, c_2)}_{q_1 \text{ 的候选块}}.$$

为了让查询 q_q 与 tile 中每个候选 c_t 都相乘，查询的整行必须连续复制 T 次：

$$A^{\text{pack}} = Q \otimes \mathbf{1}_T \in \mathbb{R}^{mT \times d}, \quad A_{qT+t, f}^{\text{pack}} = Q_{q, f}.$$

具体例子中：

$$Q = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \implies A^{\text{pack}} = \begin{bmatrix} 1 & 2 \\ 1 & 2 \\ 1 & 2 \\ 3 & 4 \\ 3 & 4 \\ 3 & 4 \end{bmatrix}_{6 \times 2}.$$

逐特征读取 A^{pack} 的列，得到真正送入 CKKS 的一维向量：

$$\mathbf{a}_0 = [1, 1, 1, 3, 3, 3]^T \in \mathbb{R}^{mT},$$

$$\mathbf{a}_1 = [2, 2, 2, 4, 4, 4]^T \in \mathbb{R}^{mT}.$$

1	1	1	3	3	3	$\leftarrow \mathbf{a}_0$
2	2	2	4	4	4	$\leftarrow \mathbf{a}_1$

查询复制的必要性。若 q_0 的特征值只出现一次，它在一次逐槽乘法中只能遇到一个候选值；复制为 T 份后，同一个查询值分别占据 $t = 0, 1, \dots, T-1$ 的位置，才可能在一次 SIMD 运算中遇到 T 个不同候选。

查询扩展的计算含义。复制发生在加密前；随后仍然只按特征加密 d 个向量 $\text{Enc}(\mathbf{a}_0), \dots, \text{Enc}(\mathbf{a}_{d-1})$ 。密文数量没有从 d 变成 dT ，只是单个密文的有效槽位从 m 增加到 mT 。

3. 候选分块的槽位扩展与对齐

B 从候选库中连续取 T 行组成第 j 个 tile:

$$C^{(j)} = C[jT : (j+1)T, :] \in \mathbb{R}^{T \times d}.$$

最后一个 tile 若只有 $v < T$ 个真实候选, 就在末尾补 $T - v$ 行零; 解包时只保留前 v 列。

在例子中, $T = 3$, 候选 tile 为

$$C^{(0)} = \begin{bmatrix} 10 & 20 \\ 30 & 40 \\ 50 & 60 \end{bmatrix}_{3 \times 2}.$$

查询打包矩阵有 mT 行, 因此候选也必须变成 mT 行。做法是把整个 tile 重复 m 次:

$$B_{\text{pack}}^{(j)} = \mathbf{1}_m \otimes C^{(j)} \in \mathbb{R}^{mT \times d}, \quad B_{qT+t, f}^{(j)} = C_{t, f}^{(j)}.$$

具体展开为:

$$B_{\text{pack}}^{(0)} = \begin{bmatrix} 10 & 20 \\ 30 & 40 \\ 50 & 60 \\ 10 & 20 \\ 30 & 40 \\ 50 & 60 \end{bmatrix}_{6 \times 2}.$$

它的逐特征明文向量为:

$$\begin{aligned} \mathbf{b}_0 &= [10, 30, 50, 10, 30, 50]^T \in \mathbb{R}^{mT}, \\ \mathbf{b}_1 &= [20, 40, 60, 20, 40, 60]^T \in \mathbb{R}^{mT}. \end{aligned}$$

查询与候选的形状对应关系。 A_{pack} 和 $B_{\text{pack}}^{(j)}$ 都是 $mT \times d$ 。在同一行 $r = qT + t$:

$$A_{r, :}^{\text{pack}} = q_q, \quad B_{\text{pack}, r, :}^{(j)} = c_{jT+t}.$$

因此第 r 行从一开始就代表同一对 (q, t) ; 不存在“查询在槽位 0、候选在槽位 100”的错位问题。

槽位 r	组合语义	$A_{r, :}^{\text{pack}}$	$B_{\text{pack}, r, :}^{(0)}$
0	(q_0, c_0)	$[1, 2]$	$[10, 20]$
1	(q_0, c_1)	$[1, 2]$	$[30, 40]$
2	(q_0, c_2)	$[1, 2]$	$[50, 60]$
3	(q_1, c_0)	$[3, 4]$	$[10, 20]$
4	(q_1, c_1)	$[3, 4]$	$[30, 40]$
5	(q_1, c_2)	$[3, 4]$	$[50, 60]$

注意: 查询是“每一行重复 T 次”; 候选是“整个 $T \times d$ tile 重复 m 次”。两种重复方向不同, 但最终行索引完全一致。

4. 逐槽乘加的正确性证明

对每个特征 f , A 加密查询列 $\text{Enc}(\mathbf{a}_f)$, B 保留候选列明文 $\mathbf{b}_f$ 。TenSEAL 执行逐槽乘法和逐槽加法:

$$\mathbf{z}^{(j)} = \sum_{f=0}^{d-1} \text{Enc}(\mathbf{a}_f) \odot \mathbf{b}_f = \text{Enc} \left(\sum_{f=0}^{d-1} \mathbf{a}_f \odot \mathbf{b}_f \right) \in \text{Enc}(\mathbb{R}^{mT}).$$

命题。 对任意查询 q 和 tile 内候选位置 t , 令 $r = qT + t$, 则

$$z_r^{(j)} = \sum_{f=0}^{d-1} Q_{q,f} C_{t,f}^{(j)} = \langle q_q, c_{jT+t} \rangle.$$

证明。 由构造式可知 $a_{f,r} = Q_{q,f}$, 同时 $b_{f,r} = C_{t,f}^{(j)}$ 。逐槽运算不会混合不同下标, 因此

$$z_r^{(j)} = \sum_f a_{f,r} b_{f,r} = \sum_f Q_{q,f} C_{t,f}^{(j)}.$$

这正是目标分数表中第 $(q, jT + t)$ 个元素。证毕。

把例子代入:

$$\begin{aligned} \mathbf{z}^{(0)} &= \text{Enc}([1, 1, 1, 3, 3, 3]) \odot [10, 30, 50, 10, 30, 50] \\ &\quad + \text{Enc}([2, 2, 2, 4, 4, 4]) \odot [20, 40, 60, 20, 40, 60] \\ &= \text{Enc}([50, 110, 170, 110, 250, 390]). \end{aligned}$$

按 query-major 规则恢复二维形状:

$$\text{reshape}_{m \times T}([50, 110, 170, 110, 250, 390]) = \begin{bmatrix} 50 & 110 & 170 \\ 110 & 250 & 390 \end{bmatrix} = QC^T.$$

项目最终还要减阈值并乘独立正随机掩码。对每个组合 (q, t) :

$$\tilde{S}_{q,t} = R_{q,t} \left(\sum_f Q_{q,f} C_{t,f} - \tau \right), \quad R_{q,t} > 0.$$

这仍然是逐槽运算, 不改变上面的槽位对应关系。

无旋转推论。 每个点积都只在固定槽位 $r = qT + t$ 内沿特征维求和; 不同槽位之间从不需要交换数据。因此打包阶段和打分阶段都不需要 slot rotation。

5. 项目两轮候选数据的构造

上面的证明把 $C^{(j)}$ 写成一个普通候选 tile，这是槽位几何的核心。在当前项目中，两轮的候选来源略有不同，但都服从同一个 $qT + t$ 布局。

第一轮候选数据为聚类中心。 B 有 $k = 50$ 个聚类中心，每个中心是 200 维特征：

$$C_{\text{centroid}} \in \mathbb{R}^{50 \times 200}.$$

每次取 $T = 20$ 个中心形成 $C^{(j)} \in \mathbb{R}^{20 \times 200}$ ，因此共有

$$\left\lceil \frac{50}{20} \right\rceil = 3 \text{ 个 tiles.}$$

这里同一组中心对所有查询相同，正好就是 $\mathbf{1}_m \otimes C^{(j)}$ 的直接情形。

第二轮候选数据为所选聚类中的候选列。 B 的库可以写成

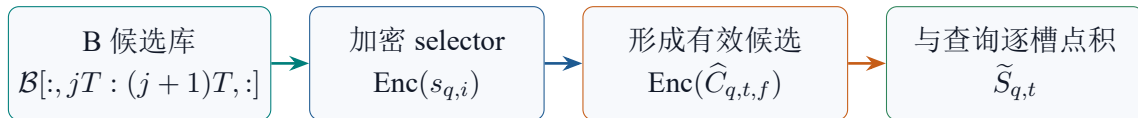
$$\mathcal{B} \in \mathbb{R}^{k \times L \times d},$$

其中 i 是聚类编号， t 是该聚类内的候选位置， f 是特征。查询 q 对应一个加密 one-hot selector $s_{q,i}$ 。因此它真正要比较的“有效候选”为

$$\hat{C}_{q,t,f} = \sum_{i=0}^{k-1} s_{q,i} \mathcal{B}_{i,t,f}.$$

第二轮候选选择细节。 第二轮并不是先把某个明文聚类公开选出来；B 将每个聚类候选的明文贡献与加密 selector 在同槽位相乘，同态形成 $\hat{C}_{q,t,f}$ 。因此不同查询可以在同一 tile 中选择不同聚类，但槽位仍然是 $qT + t$ 。随后再与 $\text{Enc}(Q_{q,f})$ 做逐槽点积。

把第二轮单个 tile 的流程写成四步：



第二轮 $L = 483$ ，于是候选位置被切为

$$[0, 19], [20, 39], \dots, [460, 479], [480, 482] + 17 \text{ 行零填充},$$

总计 $\lceil 483/20 \rceil = 25$ 个 tiles。零填充槽位在解包时丢弃，不会被解释成真实候选。

6. 计算与通信开销比较

设候选宽度为 L 、特征维度为 d 、查询数为 m 、tile 宽度为 T 。先比较最核心的“普通候选点积”部分：

项目	修改前（逐候选轮询）	修改后（二维 tile）
一次结果的逻辑形状	$m \times 1$ ，只是一列	$m \times T$ ，一次得到 T 列
查询输入	d 个长度 m 的有效向量	d 个长度 mT 的有效向量
查询密文数量	d	d ，不增加
B 端候选准备	每轮广播一个候选， $O(md)$	每个 tile 展开为 $mT \times d$ ， $O(mTd)$
在线外层循环	L 次	$\lceil L/T \rceil$ 次
密文-明文乘法量	Ld	$\lceil L/T \rceil d$
slot rotation	不需要	不需要
结果密文数	L	$\lceil L/T \rceil$

预处理开销。二维打包确实增加了加密前的槽位写入和 B 端每 tile 的明文展开：逻辑数据量从 m 增加到 mT 。但 CKKS 多项式度和安全参数保持不变，查询仍是每个特征一个密文；B 端展开在本地完成，不增加 A 到 B 的候选通信。换来的收益是：一次同态点积不再只产生 m 个分数，而是产生 mT 个分数。

当前参数代入：

$$S_{\text{slots}} = 4096, \quad m = 200, \quad T = \left\lfloor \frac{4096}{200} \right\rfloor = 20.$$

$$mT = 200 \times 20 = 4000, \quad \text{槽位利用率} = \frac{4000}{4096} = 97.66\%.$$

$$L = 483: \quad 483 \text{ 次候选轮询} \implies \left\lceil \frac{483}{20} \right\rceil = 25 \text{ 个 tiles}.$$

结论 复制不是重复执行 T 次同态计算；
它是在加密前把 $m \times T$ 个 (q, t) 组合排进同一密文，让一次逐槽乘加自然产生 mT 个分数。

因此实际加速应接近候选分块比例：

$$\frac{483}{25} = 19.32, \quad \frac{2722.17 \text{ s}}{144.42 \text{ s}} \approx 18.85.$$

二者接近，说明收益主要来自每次并行处理约 $T = 20$ 个候选，而不是修改 CKKS 算法、降低安全参数或省略原有打分步骤。